

VIGIL-X

Vigilant Industrial Guard and Intelligence Layer

DOCUMENTATION TECHNIQUE

Architecture, sécurité, IA, API, déploiement et exploitation

Projet officiel	MISSION SENTINEL-X - L'Avant-Poste Industriel du Futur
Interface / nom produit	VIGIL-X
Contexte	Workshop national EPSI BAC+4 - Octobre 2026
Version documentaire	1.0 - état validé au 08/10/2026
Architecture retenue	Edge-to-Server sur PC local, conteneurisée par Docker Compose

Document de référence pour développement, démonstration, exploitation et maintenance.

Résumé exécutif

VIGIL-X est l'implémentation de supervision du projet officiel SENTINEL-X. Le prototype agrège des données environnementales et de présence, applique une logique d'alerte locale, transporte les mesures de façon sécurisée, les persiste dans MongoDB, les analyse avec un modèle Isolation Forest et les restitue dans un dashboard Web réactif.

Transport MQTTs / TLS 8883	Authenticité HMAC-SHA256	Backend Flask + MongoDB	IA Isolation Forest	Supervision React / 1 s
--	------------------------------------	---------------------------------------	-----------------------------------	-----------------------------------

État du prototype - Le cœur Edge-to-Server est fonctionnel en simulation Wokwi. La webcam réelle et le matériel ESP8266 physique restent des écarts par rapport au cahier des charges officiel.

1. Références et périmètre

Le cahier des charges EPSI impose un système cyber-physique intégrant IoT, IA locale, infrastructure conteneurisée et cybersécurité. Le serveur local peut être un Raspberry Pi 5 ou le PC d'un apprenant. L'équipe a retenu l'option PC apprenant (topologie Edge-to-Server).

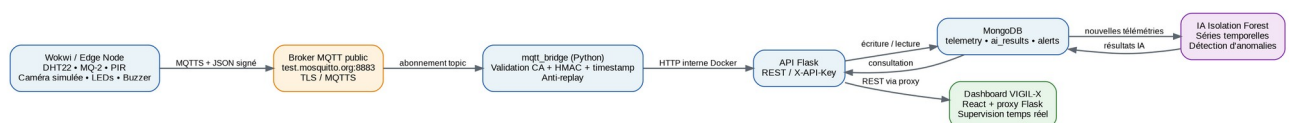
- Cible officielle : ESP8266, DHT22, MQ-2, PIR HC-SR501, OLED, buzzer, LEDs et webcam USB côté serveur.
- Stack serveur conteneurisée via Docker Compose avec base de données, API et broker MQTT.
- Chiffrement obligatoire du flux IoT par MQTTs/TLS ou HTTPS, durcissement, audit/pentest et MCO.
- IA de séries temporelles obligatoire : les simples règles statiques ne suffisent pas à elles seules.
- Dashboard temps réel et démonstration intégrée comme critère majeur de maintenance.

2. État de conformité

Exigence	Implémentation actuelle	Statut
Collecte température/humidité	DHT22 simulé sur Wokwi	Implémenté
Gaz / fumées	Entrée analogique MQ-2 simulée + calibration baseline	Implémenté
Présence	PIR simulé	Implémenté
Alertes physiques	LED verte/orange/rouge + buzzer	Implémenté
ESP8266	ESP32 Wokwi utilisé comme équivalent de simulation	Partiel
Webcam USB / vision	Champ caméra simulé par bouton ; pas de flux webcam réel	À faire
MQTT chiffré	test.mosquitto.org:8883, CA	Implémenté

Exigence	Implémentation actuelle	Statut
	vérifiée	
Authenticité des messages	HMAC-SHA256 + timestamp + anti-replay	Implémenté
API / historique	Flask REST + MongoDB	Implémenté
IA séries temporelles	Isolation Forest + lissage gaz + confirmation temporelle	Implémenté
Dashboard réactif	React, polling 1 s, timeout capteur 5 s	Implémenté
Hardening hôte / firewall	Reste à formaliser et prouver	À finaliser
Pentest croisé / rapport audit	Scénarios à exécuter et documenter	À faire

3. Architecture générale



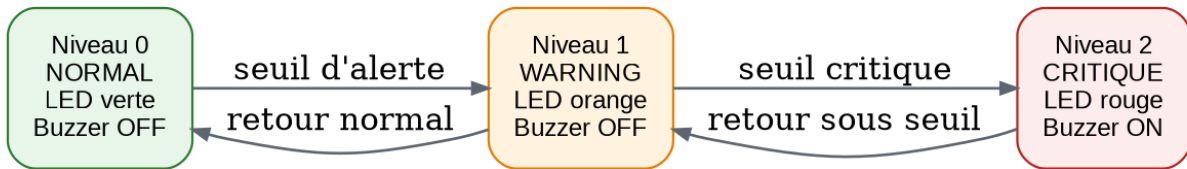
Le flux est volontairement découpé en composants spécialisés : le nœud Edge mesure et signe ; le broker transporte ; le bridge valide la sécurité ; l'API persiste ; l'IA enrichit ; le dashboard expose la vue opérateur.

4. Architecture IoT / Wokwi

Fonction	GPIO simulation	Rôle
DHT22	15	Température et humidité
PIR	13	Présence / mouvement
MQ-2 analogique	34	Gaz / fumée et baseline locale
Bouton caméra simulée	14	Simule un signal de présence caméra
Buzzer	12	Alerte sonore niveau critique
LED verte	25	Niveau 0
LED orange	26	Niveau 1
LED rouge	27	Niveau 2

Au démarrage, le firmware initialise le Wi-Fi, synchronise l'heure via NTP, charge le certificat CA, ouvre une session MQTT TLS, calibre le MQ-2 pendant environ cinq secondes, puis publie une mesure environ chaque seconde.

5. Moteur de niveaux locaux



Signal	Warning - niveau 1	Critique - niveau 2
Température	< 18 °C ou > 28 °C	< 10 °C ou > 40 °C
Humidité	< 30 % ou > 70 %	< 20 % ou > 85 %
Gaz relatif	baseline + 400	baseline + 800
Gaz absolu	>= 3000	>= 3500
Présence	PIR seul ou caméra simulée seule	PIR + caméra simulée simultanément

Important - Ces seuils pilotent l'état local et le dashboard, mais ne remplacent pas l'IA. Le modèle Isolation Forest apporte une détection comportementale sur les séries temporelles.

6. Format de télémétrie MQTT

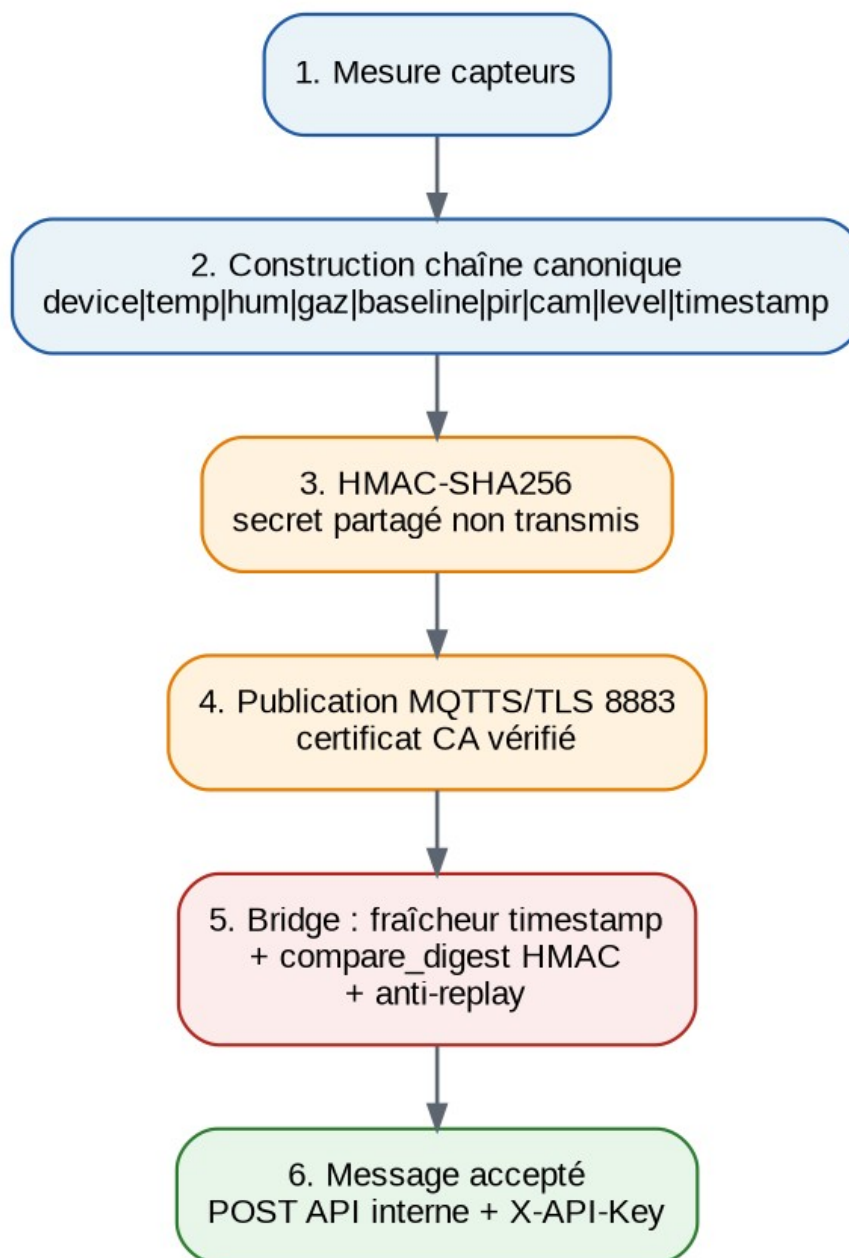
```

{
  "device_id": "esp32-wokwi",
  "temp": 27.1,
  "hum": 55.0,
  "gaz": 1843,
  "gaz_base": 2092,
  "pir": 0,
  "cam": 0,
  "level": 0,
  "timestamp": 1791463847,
  "signature": "<hmac_sha256_hex>"
}
  
```

La signature porte sur une chaîne canonique strictement identique côté firmware et côté bridge :

device_id | temp(1 décimale) | hum(1 décimale) | gaz | gaz_base | pir | cam | level | timestamp

7. Sécurité du transport et des messages



- TLS/MQTTs sur le port 8883 : confidentialité et intégrité du transport jusqu'au broker.
- Vérification du certificat CA Mosquitto : CERT_REQUIRED, validation du nom d'hôte, mode insecure désactivé.
- HMAC-SHA256 : authentifie le contenu applicatif grâce à un secret partagé qui n'est jamais transmis sur MQTT.
- Timestamp : rejette les messages trop anciens ou trop futurs. La simulation est configurée avec une fenêtre de 600 s pour tolérer la dérive Wokwi.
- Anti-replay : le bridge mémorise temporairement les signatures déjà acceptées et refuse leur réutilisation.
- NTP : synchronisation au démarrage et resynchronisation périodique côté Wokwi afin de limiter la dérive temporelle.
- API key : le bridge et le dashboard serveur utilisent X-API-Key pour appeler l'API Flask.

Limite connue - Le broker test.mosquitto.org est public et anonyme. TLS protège le transport mais

ne rend pas le topic privé. HMAC empêche néanmoins un tiers sans secret de forger un message accepté par le bridge. Pour une version production, utiliser un broker privé avec authentification et ACL.

8. mqtt_bridge

Le service Python mqtt_bridge constitue la frontière de confiance entre le broker externe et le backend interne. Il se connecte en TLS, s'abonne au topic de capteurs, decode le JSON, vérifie les champs obligatoires, contrôle la sécurité puis transmet uniquement la télémétrie métier à l'API.

Contrôle	Comportement
Secret HMAC absent	Le conteneur refuse de démarrer.
timestamp/signature absents	Message ignoré.
Âge > HMAC_MAX_AGE_SECONDS	Message rejeté.
Signature HMAC invalide	Message rejeté.
Signature déjà vue	Replay rejeté.
Message valide	POST /api/v1/telemetry avec X-API-Key.

Le bridge retire volontairement timestamp et signature avant de transmettre l'objet métier à l'API. L'API reste ainsi découplée du protocole de signature MQTT.

9. API Flask

Méthode	Route	Usage
GET	/health	Santé de l'API
GET	/api/v1/telemetry?limit=N	Historique télémétrie
POST	/api/v1/telemetry	Ingestion télémétrie validée
GET	/api/v1/alerts?limit=N	Historique alertes
POST	/api/v1/alerts	Création d'une alerte applicative
GET	/api/v1/ai-results?limit=N	Résultats IA
GET	/api/v1/status	Dernière télémétrie + dernier résultat IA

Toutes les routes métier exigent le header X-API-Key. La comparaison est réalisée avec hmac.compare_digest. Lors de l'ingestion, l'API normalise les types et ajoute received_at en UTC.

10. Modèle de données MongoDB

Collection	Contenu principal
telemetry	device_id, temperature, humidity, gas, gas_baseline, pir, camera, level, received_at
ai_results	telemetry_id, features calculées, score, model_anomaly, anomaly_streak, is_anomaly, dates
alerts	device_id, sensor, value, unit, status, received_at

MongoDB n'est pas publié directement vers l'hôte dans la configuration actuelle, ce qui limite l'exposition réseau de la base.

11. Intelligence artificielle - Isolation Forest

Le service IA charge un modèle Isolation Forest sérialisé avec joblib et surveille la collection telemetry. Chaque nouvelle mesure est transformée en vecteur de caractéristiques puis analysée.

Caractéristique IA	Traitement
Température	Valeur numérique brute
Humidité	Valeur numérique brute
Gaz	Moyenne glissante sur 5 mesures
Delta gaz IA	Neutralisé à 0 tant que la hausse n'est pas significative
Ratio gaz IA	Neutralisé à 1 tant que la hausse n'est pas significative
PIR	0/1
Caméra	0/1

- Hausse gaz considérée significative si delta ≥ 700 ou ratio $\geq 1,35$.
- Une anomalie du modèle doit être observée deux fois consécutivement pour devenir is_anomaly = true.
- Le résultat conserve également le niveau de règles locales pour comparaison et traçabilité.
- Un mécanisme anti-doublon associe chaque résultat IA à un telemetry_id unique.

12. Dashboard VIGIL-X

Le dashboard est construit en React et servi par un petit serveur Flask/Gunicorn qui joue également le rôle de proxy vers l'API interne. Le navigateur ne manipule donc pas directement l'API key.

Comportement	Description
Rafraîchissement	Polling toutes les 1 000 ms
Session propre	Les anciennes données sont marquées comme déjà vues au premier chargement et ne remplissent pas les graphes de la nouvelle session
Graphes	Maximum 30 points récents de la session
Timeout capteur	Après 5 s sans nouvelle télémétrie, les cartes reviennent à zéro et l'IA affiche -- ; les graphes restent figés
Niveau système	NORMAL / WARNING / CRITIQUE selon telemetry.level
Analyse IA	NORMAL ou ACTIVITÉ SUSPECTE selon is_anomaly

13. Orchestration Docker Compose

Service	Rôle	Exposition hôte
mosquitto	Broker local historique / actuellement non utilisé pour le flux Wokwi externe	1883:1883
mongo	Persistance	Aucune
api	API REST	5000:5000
mqtt-bridge	Ingestion sécurisée du broker public	Aucune
ai	Analyse Isolation Forest	Aucune
dashboard	Interface Web + proxy API	3000:3000

Nettoyage recommandé - Le service Mosquitto local exposé sur 1883 est actuellement inutile pour le scénario Wokwi qui utilise test.mosquitto.org. Avant rendu final, le supprimer de la stack ou le sécuriser (authentification, ACL, TLS, absence d'exposition inutile).

14. Configuration et secrets

Le fichier .env local contient les secrets et ne doit jamais être versionné. La documentation et .env.example doivent utiliser uniquement des placeholders.

```

MONGO_USER=sentinel
MONGO_PASSWORD=<mot-de-passe-fort>
API_KEY=<cle-api-aleatoire>
HMAC_SECRET=<secret-hmac-aleatoire>
HMAC_MAX_AGE_SECONDS=600

```

Rotation - Les secrets utilisés pendant les tests ou partagés à l'écran doivent être régénérés avant la remise finale.

15. Procédure de démarrage

1. Créer ou compléter .env avec des secrets valides.
2. Vérifier le certificat mqtt_bridge/mosquitto.org.crt.
3. Lancer docker compose config --quiet pour valider le YAML.
4. Construire et démarrer : docker compose up -d --build.
5. Contrôler : docker compose ps.
6. Lancer Wokwi avec le même HMAC_SECRET que le bridge (secret de test uniquement).
7. Ouvrir le dashboard sur <http://localhost:3000>.

```

docker compose config --quiet
docker compose up -d --build
docker compose ps
docker compose logs -f mqtt-bridge

```

16. Tests de validation exécutés

Test	Résultat attendu / observé
Connexion MQTTS	Connexion TLS établie sur test.mosquitto.org:8883 avec CA valide.
Message non signé	Rejet : champ timestamp/signature manquant.
Message signé valide	HMAC valide, timestamp valide, anti-replay valide, API 201 stored.
Niveau 2	Télémetries critiques stockées et visibles ; IA peut remonter ACTIVITÉ SUSPECTE.
Retour niveau 0	Le flux continue après l'événement critique.
Dérive Wokwi	Identifiée puis corrigée par fenêtre 600 s + resynchronisation NTP périodique.
Endpoints dashboard	/api/telemetry et /api/ai-results retournent des données récentes.

17. Stratégie de pentest recommandée

- Injection MQTT sans timestamp/signature : doit être refusée.
- Injection avec signature aléatoire : doit être refusée.
- Replay exact d'un ancien payload signé : doit être refusé par la cache anti-replay ou la fenêtre temporelle.
- Appel API sans X-API-Key / avec clé invalide : HTTP 401 attendu.
- Scan Nmap du PC serveur : seuls les ports nécessaires doivent rester ouverts.
- Capture Wireshark du trafic MQTT : le contenu applicatif ne doit pas être lisible en clair sur le transport TLS.
- Test de résilience : arrêt/restart d'un conteneur et vérification de la reprise du flux.

18. MCO, supervision et dépannage

Symptôme	Diagnostic	Action
Dashboard à zéro	Pas de nouvelle télémétrie depuis > 5 s	Vérifier bridge, Wokwi, timestamp et endpoints
Signature HMAC invalide	Secret différent ou canonicalisation différente	Comparer HMAC_SECRET et format 1 décimale
Message trop ancien/futur	Dérive d'horloge	Vérifier NTP et HMAC_MAX_AGE_SECONDS
API 401	API_KEY incorrecte	Vérifier .env / variables du conteneur
API 201 mais pas IA	Service ai arrêté ou modèle absent	docker compose logs ai
TLS certificate verify failed	CA absente/invalide	Vérifier mosquitto.org.crt et tls_insecure_set(False)

19. Risques, limites et dette technique

Sujet	Risque / limite	Action recommandée
Broker public	Disponibilité et anonymat hors contrôle de l'équipe	Broker privé authentifié pour version finale/production
Secret dans firmware	Extractible sur matériel ou visible dans Wokwi	Provisionnement de secret et rotation ; ne pas committer le vrai secret
Anti-replay mémoire	Cache perdue au redémarrage du bridge	Passer à compteur de séquence persistant pour robustesse supérieure
Webcam	Non implémentée	Ajouter capture USB + modèle

Sujet	Risque / limite	Action recommandée
		vision
ESP32 simulation	Différence avec cible ESP8266	Porter et tester le firmware sur ESP8266 physique
Hardening Docker	Privilèges/réseaux encore perfectibles	no-new-privileges, drop caps, réseaux internes, lecture seule si possible
README	Dépôt main actuellement sans README utile	Rédiger procédure d'installation et architecture

20. Roadmap de finalisation

8. Porter le firmware sur ESP8266 physique et valider le câblage réel.
9. Intégrer webcam USB et pipeline de vision demandé par le sujet.
10. Durcir l'hôte : UFW/iptables, accès SSH par clé, limitation des ports.
11. Durcir Docker : réseaux séparés, capacités minimales, no-new-privileges, healthchecks.
12. Exécuter le pentest croisé et documenter preuves, vulnérabilités et corrections.
13. Nettoyer le dépôt : README, .env.example, suppression de secrets et du broker local inutile si non retenu.
14. Préparer script de démo et captures de preuves de chiffrement / IA / dashboard.

Annexe A - Arborescence logique

```

workshop-M1-G5/
├── api/           # API Flask + MongoDB
├── ai/            # entraînement + détecteur Isolation Forest
├── cad/wokwi/     # simulation firmware / câblage
├── dashboard/    # React + proxy Flask/Gunicorn
├── infra/        # configuration infrastructure / Mosquitto
├── mqtt_bridge/  # client MQTT TLS + HMAC + forwarding API
├── security/     # livrables de sécurité à compléter
├── docker-compose.yml
├── .env.example
└── README.md

```

Annexe B - Sources de référence

- EPSI - Sujet de Workshop National BAC+4 2026-27 : Mission SENTINEL-X.
- Dépôt GitHub Enlight-stack/workshop-M1-G5, branche main, état consulté le 08/10/2026.
- Résultats de tests d'intégration Wokwi / MQTT / HMAC / API / IA / dashboard réalisés le 08/10/2026.